

Tecnologías usadas - nivel técnico

Proyecto: Zaino Web / Cuaderno de Campo Digital

Documento técnico generado a partir de los archivos del workspace. Resume la arquitectura, frameworks, dependencias y puntos de integración visibles en el código.

1. Resumen de arquitectura

El repositorio combina dos aplicaciones web con responsabilidades distintas: una capa principal en Flask para el panel operativo y generación de informes, y un módulo Django independiente para gestión de usuarios, roles, permisos y persistencia en PostgreSQL. En el frontend predominan HTML5, CSS3 y JavaScript, con Bootstrap 5, Bootstrap Icons y jQuery como base de interfaz. La aplicación también integra exportación de reportes del lado cliente mediante html2canvas y jsPDF.

2. Backend

Tecnología	Uso técnico en el proyecto
Python 3.8+	Lenguaje base de ambos subsistemas.
Flask 3.1.2	Servidor principal del panel web y endpoints API en app.py y app/controllers/*.
Django 5.2	Subaplicación <code>cuaderno_campo_django</code> para autenticación, roles, permisos y CRUD.
Django ORM	Modelo relacional y acceso a PostgreSQL desde <code>usuarios/models.py</code> .
PostgreSQL	Base de datos principal del módulo Django.
requests	Cliente HTTP usado para integrar servicios externos como Weathercloud.
requests-oauthlib / oauthlib	Autenticación basada en OAuth2 para integraciones externas.
python-dotenv / dotenv	Carga de variables de entorno desde <code>.env</code> .
filelock	Sincronización/lock de archivos en escenarios concurrentes.

3. Frontend

Tecnología	Uso técnico en el proyecto
HTML5	Plantillas principales y renderizado del layout.
CSS3	Estilos base, layout responsive y variables de tema en <code>app/static/css/app.css</code> .
JavaScript (ES6+)	Lógica de UI, navegación dinámica y consumo de APIs en <code>app/static/js/app.js</code> .
jQuery 3.6	Manipulación del DOM y helpers de compatibilidad.
Bootstrap 5.3	Sistema de grillas, modales, botones y componentes UI.
Bootstrap Icons	Iconografía vectorial de la interfaz.
html2canvas 1.4.1	Captura de vistas HTML para exportación a imagen/PDF.
jsPDF 2.5.1	Generación de PDF en el navegador sin pasar por backend.

4. Integraciones y servicios externos

El proyecto integra Arduino IoT Cloud para datos de caudal y Weathercloud para datos meteorológicos. La clase `WeathercloudAPI` usa `requests.Session`, maneja cookies y ejecuta peticiones POST hacia endpoints privados simulando el flujo del navegador. En el HTML principal también se consumen recursos CDN de Bootstrap, Bootstrap Icons, jQuery, html2canvas y jsPDF.

5. Persistencia, seguridad y control de acceso

El módulo Django usa PostgreSQL como motor de persistencia, middleware propio para control de acceso por rol y templates con contexto dinámico. En `settings.py` se configuran cookies de sesión, CSRF, `X_FRAME_OPTIONS` en

ALLOWALL para convivencia con Flask embebido e integraci3n por iframe en escenarios locales.

6. Exportaci3n y reportes

La generaci3n de reportes se apoya en reportlab para PDF del lado servidor y openpyxl/pandas/matplotlib para exportaci3n y an3lisis de datos. En el frontend, la exportaci3n visual usa html2canvas + jsPDF, evitando procesamiento adicional en el backend para vistas simples.

7. Dependencias observadas

Dependencias principales listadas en requirements.txt y cuaderno_campo_django/requirements.txt: Flask, Django, requests, oauthlib, requests-oauthlib, python-dotenv, psycpg2-binary, openpyxl, reportlab, pandas y matplotlib.

8. Conclusi3n t3cnica

El proyecto no es monol3tico: est3 dividido en una app Flask de presentaci3n/operaci3n y un subsistema Django para administraci3n y seguridad. La capa de UI es mayormente server-rendered, pero con una l3gica de interacci3n rica en JavaScript que actualiza datos por fetch y permite exportaci3n PDF en cliente.